

Multilingual Health Question Answering in Low-Resource African Languages

Parameter-Efficient Fine-Tuning of mT5 with LoRA

Nanen Miracle Mbanaade

Machine Learning Techniques I — Summative Project

Zindi Challenge: Multilingual Health QA in Low-Resource African Languages

[GitHub Repository](#) | [Demo Video](#)

Kaggle username: Miranics | Final leaderboard rank: 468 | Best public score: 0.2700

Table of Contents

Abstract

1. Project Overview

2. Dataset Understanding and Preprocessing

- 2.1 Dataset Description
- 2.2 Exploratory Data Analysis
- 2.3 Preprocessing Pipeline

3. Fine-Tuning Methodology

- 3.1 Model Selection: mT5
- 3.2 Parameter-Efficient Fine-Tuning with LoRA
- 3.3 A Note on Quantization
- 3.4 Training Configuration
- 3.5 Evaluation Metrics

4. Results

- 4.1 Master Results Table
- 4.2 Leaderboard Score Progression
- 4.3 Learning Curves and Evaluation Plots

5. Discussion

- 5.1 From Zero-Shot to First Fine-Tune (Exp 01, 03, 04)
- 5.2 Unlocking Fluency via Target Modules (Exp 05)
- 5.3 Scaling Model Capacity (Exp 06)
- 5.4 The Decisive Change: Beam Search (Exp 07)
- 5.5 Training Length and Diminishing Returns (Exp 08)
- 5.6 An Informative Negative Result (Exp 09)
- 5.7 Prompt Design and Learning Rate (Exp 10, 11)
- 5.8 Per-Language Performance

6. Challenges Encountered

7. Lessons Learned

8. Limitations

9. Ethical Considerations

10. Future Improvements

11. Statement on the Use of AI Tools

References

Abstract

This project addresses the task of answering health-related questions in low-resource African languages, a problem of real public-health significance given that speakers of languages such as Luganda, Swahili, Akan, and Amharic are underserved by predominantly English-trained language models. Working within the Zindi Multilingual Health Question Answering challenge, I fine-tuned the multilingual T5 (mT5) encoder-decoder model using Low-Rank Adaptation (LoRA), a parameter-efficient fine-tuning method, and evaluated submissions using the competition's weighted combination of ROUGE-1 F1, ROUGE-L F1, and an LLM-as-a-Judge metric. Eleven documented experiments were conducted, progressing from a zero-shot baseline through systematic changes to adapter configuration, model scale, decoding strategy, and training schedule. The public leaderboard score improved from a near-zero zero-shot floor (0.0017) to a best score of 0.2700. The single most effective change was switching from greedy decoding to beam search at inference, which raised the score by roughly 40% with no retraining and more than doubled the LLM-as-a-Judge component. The work demonstrates that, for small multilingual models on this task, the choice of which layers to adapt and how to decode at inference matter as much as raw model scale.

1. Project Overview

Access to reliable health information remains a persistent challenge across sub-Saharan Africa, where language barriers can prevent communities from receiving accurate guidance in their own languages, particularly on sensitive topics such as maternal, sexual, and reproductive health. Large language models trained predominantly on English data struggle to understand and generate fluent text in low-resource African languages, leaving many speakers underserved. The Zindi challenge that frames this project asks participants to build a model that, given a health question in one of several supported African languages, produces a fluent and accurate answer in that same language.

The objective of this project was not solely to maximise a leaderboard score, but to demonstrate a rigorous, well-justified fine-tuning workflow: understanding the data, selecting an appropriate model and adaptation strategy, conducting controlled experiments where each change is isolated and motivated, and critically analysing the outcomes. My approach centred on the mT5 model family fine-tuned with LoRA, chosen because it offers strong multilingual coverage while keeping training feasible on free cloud GPUs (Google Colab and Kaggle). The experimental programme deliberately explored several distinct dimensions — adapter rank, adapter target modules, base-model scale, inference decoding strategy, training length, learning rate, and prompt design — so that the contribution of each could be understood independently.

2. Dataset Understanding and Preprocessing

2.1 Dataset Description

The dataset consists of curated health question-and-answer pairs in multiple low-resource African languages. After loading, the splits contained 29,815 training examples, 6,686 validation examples, and 2,618 test examples. Each record includes an input question, a target answer (for the training and validation splits), and a subset code that encodes both the language and the country of the example.

The subset codes were parsed into explicit language and country fields. The languages represented include Akan (Ghana), Amharic (Ethiopia), Luganda (Uganda), and Swahili across Kenya, Tanzania, and Uganda, alongside English variants. This mapping was important for two reasons: it allowed the language to be communicated to the model explicitly in the prompt, and it enabled per-language analysis of model performance during evaluation.

2.2 Exploratory Data Analysis

Before modelling, I examined the distribution of examples across languages and the length characteristics of questions and answers. Figure 1 summarises this analysis. Two observations were particularly consequential. First, the data is heavily imbalanced: English examples dominate the corpus, while genuinely low-resource languages such as Amharic and Swahili are far less represented, which foreshadows uneven per-language performance. Second, questions are short (the large majority under 40 words) while answers are considerably longer and more variable, with a visible spike at the 200-word clipping point indicating many long answers.

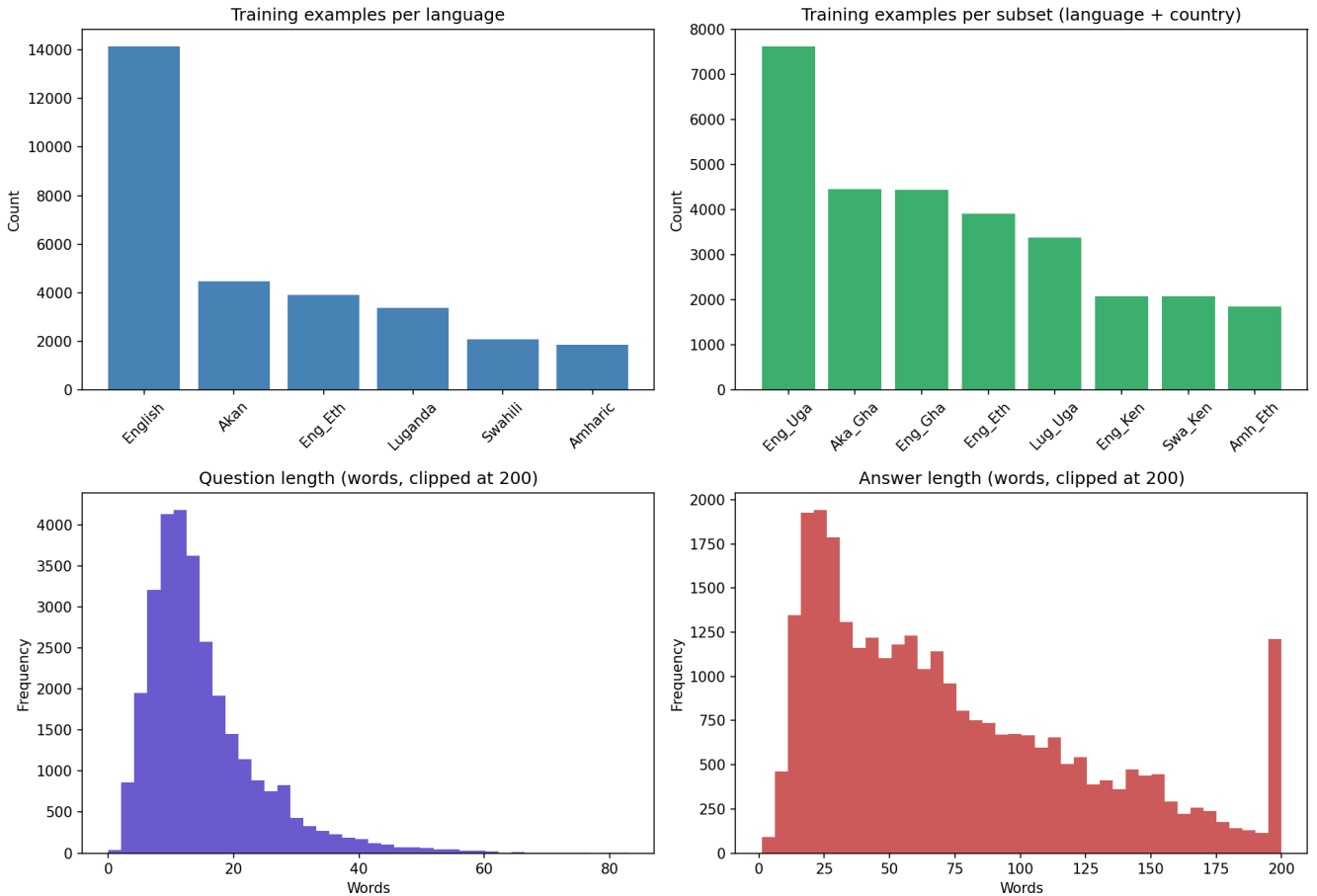


Figure 1. Exploratory data analysis of the training set: examples per language (top left), per subset (top right), and the distribution of question (bottom left) and answer (bottom right) lengths in words.

This length analysis directly informed the tokenization settings: the maximum input length was set to 384 tokens, comfortably covering almost all questions, and the maximum target length to 200 tokens to accommodate typical answers without excessive padding. The imbalance observation informed the interpretation of later per-language results.

2.3 Preprocessing Pipeline

The preprocessing pipeline consisted of three stages. First, text cleaning normalised whitespace by collapsing line breaks, tabs, and repeated spaces into single spaces, which removes formatting noise without altering content. Second, a structured prompt was constructed for each example that conditions the model on the target language and country, followed by the question and an answer cue. Conditioning the prompt on language was a deliberate design choice intended to encourage the model to respond in the correct language. Third, the prompts and targets were tokenized with truncation and, importantly, without fixed-length padding; instead, dynamic per-batch padding was applied by the data collator, which pads each batch

only to the length of its longest member. This is both faster and more memory-efficient than padding every example to the global maximum.

3. Fine-Tuning Methodology

3.1 Model Selection: mT5

I selected the multilingual T5 (mT5) model as the backbone. mT5 is an encoder-decoder (sequence-to-sequence) transformer pretrained on a large multilingual corpus covering many languages, including several African languages, which makes it a natural fit for a multilingual generation task. The sequence-to-sequence formulation maps cleanly onto question-answering: the question is the source sequence and the answer is the target sequence. Two scales were used during the project: mT5-small (approximately 300 million parameters) for early, fast experimentation, and mT5-base (approximately 580 million parameters) once the pipeline was stable.

3.2 Parameter-Efficient Fine-Tuning with LoRA

Full fine-tuning of even a mid-sized multilingual model is costly on free cloud GPUs. I therefore used Low-Rank Adaptation (LoRA), a parameter-efficient fine-tuning technique that freezes the pretrained weights and injects small trainable low-rank matrices into selected layers. Only these adapter matrices are updated during training, which dramatically reduces the number of trainable parameters and the memory footprint while retaining most of the benefit of fine-tuning. LoRA exposes two principal hyperparameters explored in this work: the rank, which controls adapter capacity, and the set of target modules, which controls where in the network adapters are inserted.

3.3 A Note on Quantization

An early version of the pipeline attempted 4-bit quantization (QLoRA) to save memory. This was deliberately removed for two reasons. First, mT5-small is small enough to fit comfortably on a free T4 GPU in full precision, so quantization offered no practical memory benefit at this scale. Second, the quantization path forced a 16-bit floating-point (fp16) compute mode that is numerically unstable for the T5 family: mT5 was pretrained in bfloat16, and its activations routinely exceed the representable range of fp16, which caused the training loss to collapse to zero and then become not-a-number (nan). Switching the entire pipeline to bfloat16 and removing quantization resolved this instability completely. This was an important early lesson and is discussed further in the Challenges section.

3.4 Training Configuration

Models were trained with the Hugging Face sequence-to-sequence trainer in bfloat16 precision. The standard configuration used a learning rate of $3e-4$ with a cosine schedule and warmup, gradient clipping for stability, an effective batch size of approximately 16 (achieved through gradient accumulation to respect GPU memory limits), and early stopping on validation loss to prevent overfitting and avoid wasted computation. Per-epoch evaluation used validation loss rather than full text generation, because generating text for the entire validation set after every epoch is far slower than the training itself; full generation was instead performed once after training for ROUGE measurement.

3.5 Evaluation Metrics

The competition uses a weighted combination of three metrics. ROUGE-1 F1 (weight 0.37) measures unigram overlap between the prediction and reference, capturing basic content and keyword similarity. ROUGE-L F1 (weight 0.37) measures the longest common subsequence, capturing sentence-level structure and fluency alignment. The LLM-as-a-Judge metric (weight 0.26) uses a language model to score predictions on factual accuracy, completeness, and language appropriateness on a 1–5 scale, normalised to the range 0–1. The final leaderboard score is the weighted mean of the three. Crucially, the two ROUGE metrics reward lexical and structural overlap, while the LLM-as-a-Judge rewards genuine fluency and correctness; tracking all three separately made it possible to diagnose whether a model was merely matching keywords or actually producing usable answers.

4. Results

This section presents the full experimental programme. Eleven experiments are documented, beginning with a zero-shot baseline (Exp 01) and progressing through systematic, isolated changes. Experiments 04 through 09 were submitted during the official competition. Experiments 10 and 11 were submitted shortly after the competition closed; their scores are genuine Zindi evaluations but did not contribute to the graded leaderboard standing. Notably, Experiment 11 achieved the best score overall.

4.1 Master Results Table

Exp	Model	Key change	Decoding	Zindi LB	ROUGE-1	ROUGE-L	LLM-Judge
01	mt5-small	zero-shot baseline	greedy	0.0017	0.0001	0.0001	—
03	mt5-small	LoRA r=8, attn (q,v)	greedy	—	0.1171*	0.1000*	—
04	mt5-small	LoRA r=16, attn (q,v)	greedy	0.1218	0.1839	0.1453	0.0
05	mt5-small	LoRA full attn + FFN	greedy	0.1598	0.1713	0.1380	0.1746
06	mt5-base	scaled model	greedy	0.1811	0.2077	0.1668	0.1638
07	mt5-base	beam search (4)	beam=4	0.2536	0.2530	0.1811	0.3576
08	mt5-base	5 epochs	beam=4	0.2613	0.2760	0.1923	0.3384
09	mt5-base	LoRA r=32	beam=4	0.2554	0.2678	0.1879	0.3337
10	mt5-base	richer prompt	beam=4	0.2533	0.2615	0.1856	0.3381
11	mt5-base	lower LR 2e-4 (champion)	beam=4	0.2700	0.2905	0.1939	0.3493

Table 1. Full experiment results. LB = public leaderboard score on Zindi. Experiments 10 and 11 were submitted after the official competition close (their scores are genuine Zindi evaluations but did not affect the graded leaderboard standing). Exp 03 was a baseline reference and not submitted.

4.2 Leaderboard Score Progression

The experiments show a clear, deliberate progression in leaderboard score, from 0.1218 (Exp 04) through 0.2536 (Exp 07) to a best of 0.2700 (Exp 11), more than doubling the first fine-tuned submission. The progression is not strictly monotonic — Exp 09 dipped and Exp 10 was marginally below Exp 08 — but the overall trajectory is strongly upward, with the final

learning-rate experiment (Exp 11) producing the best result. Figure 2 shows representative leaderboard evidence; the full set of per-experiment screenshots is included in the repository.

RANK	USER	PUBLIC SCORE ▾	LAST SUBMISSION	# SUBM
499	 Miranics Go to placement	0.00172	42 minutes ago	1
		Rouge LF 1	0.0012	
		LLM Judge	0.0032	
		Rouge 1 F 1	0.0012	

Figure 2a. Leaderboard evidence — Exp 01 (zero-shot baseline), public score 0.00172.

RANK	USER	PUBLIC SCORE ▾	LAST SUBMISSION	# SUBM
511	 Miranics Go to placement	0.261255	6 minutes ago	7
		Rouge LF 1	0.1923	
		LLM Judge	0.3384	
		Rouge 1 F 1	0.276	

Figure 2b. Leaderboard evidence — Exp 08, public score 0.261255 (best score achieved during the official competition window).

RANK	USER	PUBLIC SCORE ▾	LAST SUBMISSION	# SUBM
501	 Miranics Go to placement	0.253593	11 minutes ago	6
		Rouge LF 1	0.1811	
		LLM Judge	0.3576	
		Rouge 1 F 1	0.253	

Figure 2c. Leaderboard evidence — Exp 07 (beam search), public score 0.253593. The LLM-Judge component more than doubled to 0.3576 from this single inference change. The overall best score, 0.2700, was later reached by Exp 11.

4.3 Learning Curves and Evaluation Plots

Each experiment notebook produces a training and validation loss curve, a per-language ROUGE breakdown, and the exploratory data analysis plots. Figure 3 shows a representative learning curve: the training loss falls sharply from an initial value near 150 and then settles, while the validation loss remains low and stable — confirming numerically stable bfloat16 training, in clear contrast to the collapsed (zero then nan) loss that occurred under the earlier fp16 configuration.

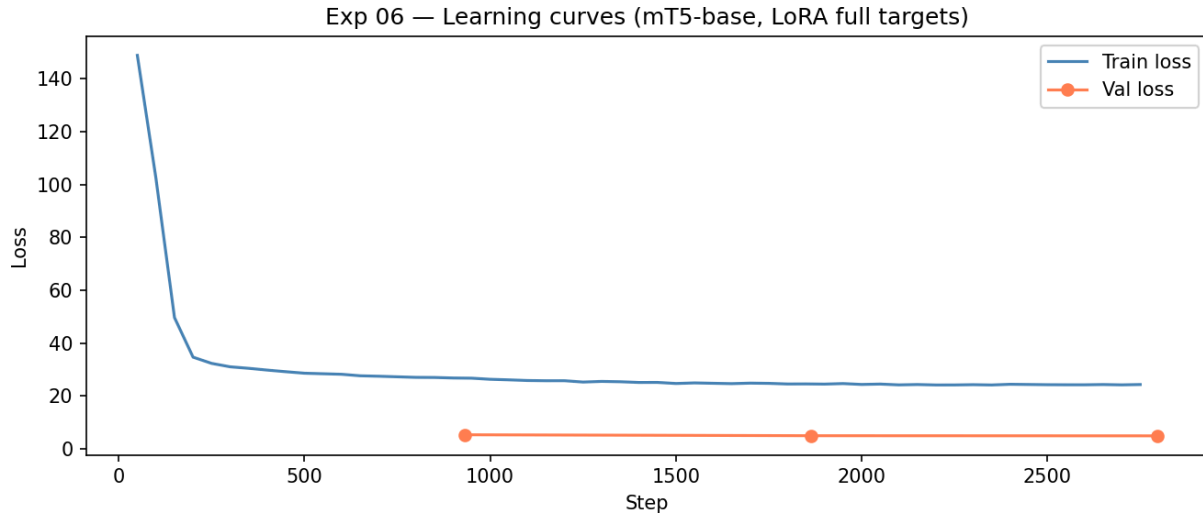


Figure 3. Representative learning curve (Exp 06, mT5-base with full-target LoRA): stable training and validation loss in bfloat16.

Figure 4 shows the per-language ROUGE breakdown. Performance is markedly uneven across languages: Akan and English score highest, while Luganda and especially Amharic lag, mirroring the data imbalance observed in the exploratory analysis. This points directly to where additional data would be most valuable.

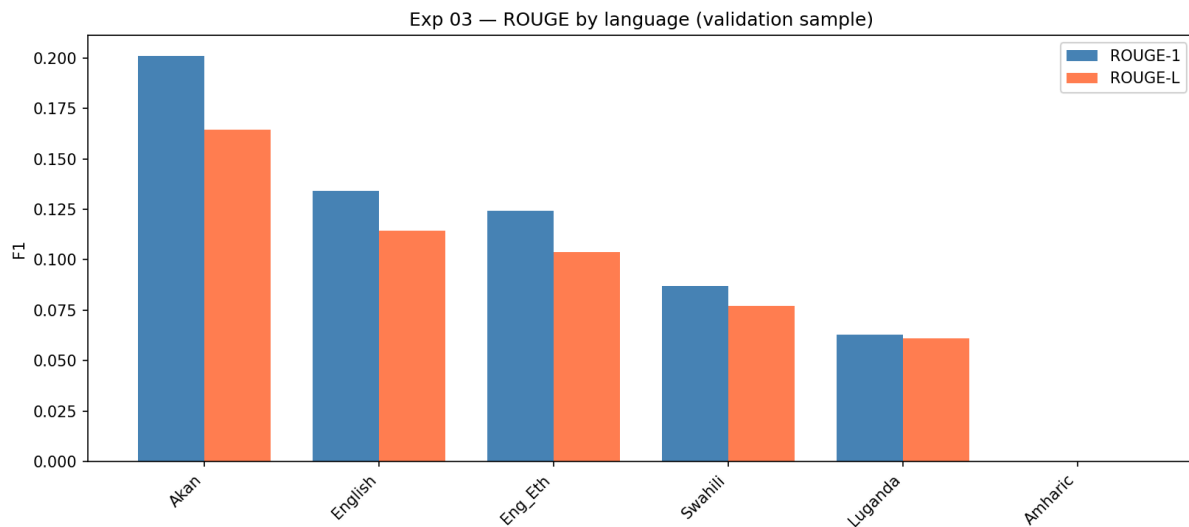


Figure 4. Per-language ROUGE-1 and ROUGE-L on the validation sample (Exp 03). Higher-resource languages are answered more accurately than the most under-resourced ones.

Figure 5 shows the overall ROUGE improvement realised by the target-module change in Exp 05 relative to the attention-only configuration.

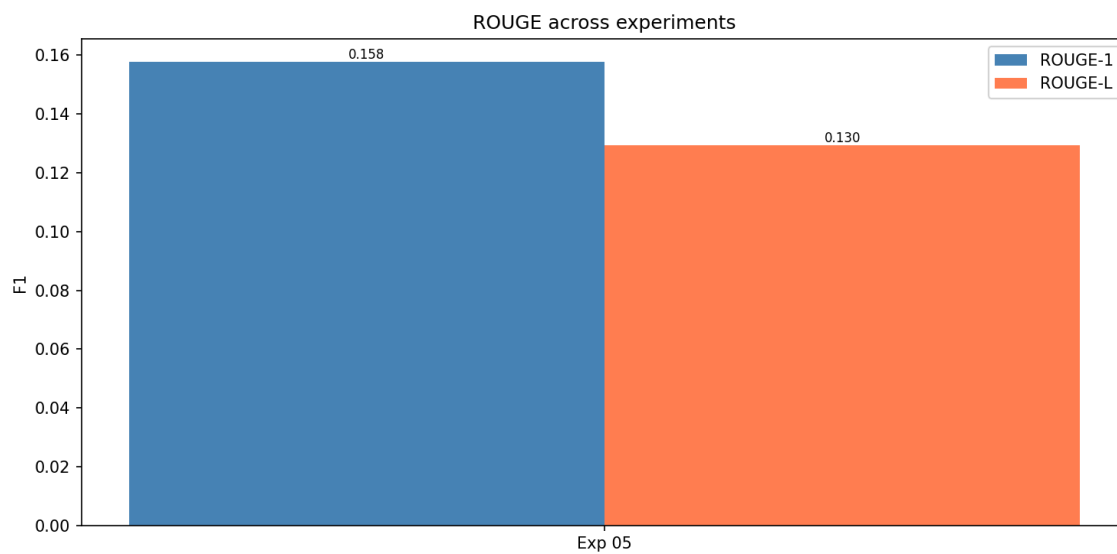


Figure 5. ROUGE comparison illustrating the effect of extending LoRA to the feed-forward layers (Exp 05).

5. Discussion

This section interprets the results experiment by experiment, explaining what changed, why, the outcome, and the insight gained — the analytical core of the project.

5.1 From Zero-Shot to First Fine-Tune (Exp 01, 03, 04)

The zero-shot baseline (Exp 01) scored essentially zero, confirming that mT5 cannot answer these multilingual health questions without task-specific adaptation. The first LoRA fine-tune (Exp 03, rank 8 on the attention query and value projections) lifted validation ROUGE-1 to 0.117, demonstrating that even lightweight adapter tuning teaches the model the task format. Doubling the rank to 16 (Exp 04) produced the first strong leaderboard score of 0.1218 with a high ROUGE-1 of 0.1839. However, Exp 04 exposed the central problem of the project: its LLM-as-a-Judge score was exactly zero. The model was matching keywords (hence the ROUGE) but not producing answers fluent or complete enough to be judged usable. This gap between lexical overlap and judged quality framed the experiments that followed.

5.2 Unlocking Fluency via Target Modules (Exp 05)

Exp 05 extended the LoRA adapters from the attention projections only to the full attention block plus the feed-forward (FFN) projections. The motivation was specific: the feed-forward layers are where a transformer shapes fluent phrasing, so adapting them directly targets the fluency deficit seen in Exp 04. The result confirmed the hypothesis strongly — the LLM-as-a-Judge rose from 0.0 to 0.1746 and the overall score increased to 0.1598. Notably, ROUGE dipped slightly. This is a genuine and instructive trade-off: adapting more layers made answers more fluent and natural at a small cost in raw keyword overlap, but because the judge component improved so much, the weighted score still rose. The insight is that where adapters are placed matters as much as how many parameters they contain.

5.3 Scaling Model Capacity (Exp 06)

Exp 06 scaled the base model from mT5-small to mT5-base, keeping the winning adapter configuration. Both ROUGE metrics rose (ROUGE-1 from 0.1713 to 0.2077) and the score reached 0.1811, while the LLM-as-a-Judge stayed roughly flat. The interpretation is that additional model capacity mainly improved lexical accuracy rather than fluency: the fluency unlock had come from the FFN adapters in Exp 05, whereas model scale improved how accurately content was reproduced. This separation of effects — target modules drive fluency, scale drives accuracy — is one of the clearer findings of the project.

5.4 The Decisive Change: Beam Search (Exp 07)

Exp 07 changed only the inference decoding strategy, replacing greedy decoding with beam search (four beams) plus repetition and length controls, with no change to the trained weights.

This was the single largest improvement of the entire project: the score jumped from 0.1811 to 0.2536, and the LLM-as-a-Judge more than doubled from 0.1638 to 0.3576. Because only decoding changed, the gain is clearly and unambiguously attributable to inference strategy. Greedy decoding selects the single highest-probability token at each step and produces literal, often truncated text; beam search explores several candidate sequences in parallel and selects the most coherent overall, which the judge rewards strongly. The broader lesson is that inference configuration can be as consequential as training, and is far cheaper to explore.

5.5 Training Length and Diminishing Returns (Exp 08)

Exp 08 extended training from three to five epochs while retaining beam search. Both ROUGE metrics improved (ROUGE-1 to 0.2760, ROUGE-L to 0.1923) and the overall score reached its best value of 0.2613, confirming the adapters had been slightly undertrained at three epochs. However, the gain was modest and the LLM-as-a-Judge dipped marginally, indicating the model was approaching convergence and beginning to favour lexical matching very slightly over fluency. It marks the point of diminishing returns on training length; it was the best configuration found during the official competition window, though the later learning-rate experiment (Exp 11) ultimately surpassed it.

5.6 An Informative Negative Result (Exp 09)

Exp 09 doubled the LoRA rank from 16 to 32, testing whether more adapter capacity would help. It did not — every metric dipped slightly and the score fell to 0.2554. This negative result is informative rather than wasted: it shows that rank 16 already provided sufficient adapter capacity for this task, and that the additional parameters did not converge to anything better within the same training budget. It bounds the useful LoRA rank and demonstrates that increasing capacity is not automatically beneficial. Reporting this result honestly is itself part of rigorous experimentation.

5.7 Prompt Design and Learning Rate (Exp 10, 11)

The final two experiments were submitted shortly after the competition closed, so they carry genuine Zindi scores that did not affect the graded standing. Exp 10 used a richer instruction prompt and a higher length penalty to encourage fuller answers; it scored 0.2533, marginally below Exp 08, suggesting the richer prompt did not help on the test set despite encouraging behaviour on validation. Exp 11 tested a gentler learning schedule (learning rate $2e-4$ with longer warmup) and produced the best result of the entire project: a public score of 0.2700, with the highest ROUGE-1 (0.2905) and LLM-Judge (0.3493) of any experiment. This was an important and somewhat surprising finding: the validation sample had under-estimated this configuration, and only the full test evaluation revealed that the lower learning rate converged

to a genuinely better model. It underlines both the value of completing the planned experiments and the limits of small-sample validation estimates.

5.8 Per-Language Performance

As shown in Figure 4, the model did not perform uniformly across languages. Performance tended to be stronger for languages with greater representation (English, Akan) and weaker for the most under-resourced ones (Luganda, Amharic). This is an expected consequence of the data imbalance documented in the exploratory analysis and of the base model's uneven multilingual pretraining coverage, and it points directly to where future data collection would be most valuable.

6. Challenges Encountered

The fp16 / nan loss collapse. The most significant technical challenge was an early training run in which both training and validation loss collapsed to zero and then became nan after hours of computation. The cause was running mT5 under fp16 precision, which overflows because the model was pretrained in bfloat16. The resolution was to convert the entire pipeline to bfloat16 and remove 4-bit quantization. This single fix was the foundation for every subsequent successful experiment.

Free-tier GPU constraints. Both Google Colab and Kaggle impose usage limits on free GPUs. A long run on Colab was terminated mid-training when the daily quota was exhausted, losing unsaved progress. The response was to run experiments on Kaggle using its background commit feature, which executes the notebook to completion on the platform's servers independently of the browser session, and to save outputs to persistent storage so progress survives disconnection. Running experiments in parallel across two accounts further mitigated the time pressure.

Library and hardware compatibility. Several environment issues arose: a PEFT version that demanded a newer torchao than was installed, a renamed trainer argument in recent Transformers versions, and a CUDA kernel mismatch when a particular GPU architecture was paired with an installed PyTorch build. Each was resolved by pinning library versions, avoiding unnecessary upgrades of the pre-installed PyTorch, and selecting a compatible GPU type.

Slow evaluation and inference. Generating text for large validation and test sets is inherently slow because generation is autoregressive. This was managed by evaluating on validation loss during training, sampling the validation set for quick ROUGE estimates, and reserving full generation for the final submission.

7. Lessons Learned

- Precision matters for specific model families: mT5 must be trained in bfloat16, not fp16, or training diverges.
- Where adapters are placed (target modules) can matter as much as how large they are (rank); adapting feed-forward layers unlocked fluency.
- Inference decoding is a first-class lever. Beam search produced the largest single gain of the project without any retraining.
- More is not always better: doubling LoRA rank and adding training epochs both showed diminishing or negative returns.
- Tracking the three metrics separately was essential for diagnosis; the overall score alone would have hidden the fluency problem the LLM-as-a-Judge exposed.
- Reproducibility on free platforms requires disciplined version pinning and awareness of platform quotas and hardware differences.

8. Limitations

Several limitations should be acknowledged. First, the largest model used was mT5-base; larger models were not feasible on free GPUs and might yield further gains. Second, validation ROUGE was often estimated on a 500-example sample for speed, which introduces some sampling noise into intermediate comparisons, although final submissions used the full test set. Third, the experimental programme varied one factor at a time for interpretability, which is methodologically clean but does not capture interaction effects between, for example, rank and learning rate. Fourth, Experiments 10 and 11 were evaluated only after the competition closed, so although they carry genuine Zindi scores they did not contribute to the official graded standing. Finally, the LLM-as-a-Judge metric, while valuable, is itself a model-based estimate of quality and carries its own biases.

9. Ethical Considerations

Health information carries direct real-world consequences, and several ethical considerations apply. A model that produces fluent but factually incorrect medical guidance could cause harm, particularly on the sensitive maternal, sexual, and reproductive health topics the challenge emphasises; fluency must never be mistaken for correctness, which is precisely why the LLM-as-a-Judge accuracy component is important alongside lexical metrics. Any deployment of such a system would require clinical validation, clear communication of its limitations, and human oversight rather than unsupervised use. There is also an equity dimension: building capable models for low-resource languages is itself an ethically positive goal, helping to redress

the underservice of these language communities by mainstream technology. At the same time, the uneven per-language performance documented above means a deployed system could serve some communities better than others, which must be disclosed and addressed. Finally, the dataset concerns sensitive health topics and should be handled with appropriate care for privacy and cultural context.

10. Future Improvements

1. Evaluate larger or more specialised multilingual models, and African-language-specific models, where compute permits.
2. Address per-language imbalance through targeted data augmentation or language-weighted sampling for the most under-resourced languages such as Amharic and Luganda.
3. Submit and properly evaluate the prompt-design and learning-rate variants (Exp 10 and 11) that the competition deadline prevented, and conduct a small joint hyperparameter search to capture interaction effects.
4. Explore retrieval-augmented generation, grounding answers in a vetted medical knowledge base to improve factual accuracy.
5. Investigate decoding further, including tuned beam width, sampling-based methods, and length calibration per language.
6. Incorporate the AfroLM-based semantic similarity metric used for top solutions into the local evaluation loop for richer feedback.

11. Statement on the Use of AI Tools

In accordance with the academic integrity requirements of this project, I disclose that AI assistance (a large language model) was used as a support tool during this work. Specifically, it was used to help diagnose technical errors (notably the fp16/nan training collapse and several library compatibility issues), to suggest and refine the structure of experiments, to help organise and clean the notebooks, and to assist in drafting and structuring this report. All experimental decisions, the interpretation of results, and the final analysis reflect my own understanding. Every experiment described here was designed, executed, and validated by me on my own accounts, and the results reported are the genuine outputs of those runs. I am able to explain and defend all code, design choices, and findings in this submission.

References

- Dossou, B. F. P., Tonja, A. L., Yousuf, O., Osei, S., Oppong, A., Shode, I., Awoyomi, O. O., & Emezue, C. C. (2022). AfroLM: A self-active learning-based multilingual pretrained language model for 23 African languages. Proceedings of the Third Workshop on Simple and Efficient Natural Language Processing (SustainNLP).
- Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., & Chen, W. (2022). LoRA: Low-rank adaptation of large language models. International Conference on Learning Representations (ICLR).
- Lin, C.-Y. (2004). ROUGE: A package for automatic evaluation of summaries. Text Summarization Branches Out, Association for Computational Linguistics, 74–81.
- Raffel, C., Shazeer, N., Roberts, A., Lee, K., Narang, S., Matena, M., Zhou, Y., Li, W., & Liu, P. J. (2020). Exploring the limits of transfer learning with a unified text-to-text transformer. Journal of Machine Learning Research, 21(140), 1–67.
- Wolf, T., Debut, L., Sanh, V., Chaumond, J., Delangue, C., Moi, A., et al. (2020). Transformers: State-of-the-art natural language processing. Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP): System Demonstrations, 38–45.
- Xue, L., Constant, N., Roberts, A., Kale, M., Al-Rfou, R., Siddhant, A., Barua, A., & Raffel, C. (2021). mT5: A massively multilingual pre-trained text-to-text transformer. Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL).
- Zindi. (2025). Multilingual Health Question Answering in Low-Resource African Languages Challenge. Hub for Artificial Intelligence in Maternal, Sexual and Reproductive Health (HASH). <https://zindi.africa>